Backstage

Practice loud. Perform proud.

Project Report and Development Journal

Field	Details
Project	Backstage — Public Speaking Training App
Project No.	8
Motto	Practice loud. Perform proud.
Teams	Frontend Group 7 and Backend Group 8
Report type	Complete project report, journal, decisions, challenges, solutions, tools and verification
Report date	27 October 2026
Current status	Backend and frontend implemented; documentation and deployment handoff prepared
Main technologies	React/Vite frontend, Django REST backend, faster-whisper/mock speech pipeline

1. Executive summary

Backstage is a private rehearsal space for people preparing speeches, presentations, interviews, debates, and demos. A user selects a prompt, records or uploads a speech, and receives feedback based on the transcript and audio timing: words per minute, pace status, pauses, filler words, timestamps, and progress across attempts.

The delivered system contains a React/Vite frontend and a Django REST backend. Authentication uses JWT. The backend accepts audio asynchronously, transcribes it with faster-whisper when available, falls back to an explicitly labelled mock engine when Whisper is unavailable, and stores the transcript plus analysis while deleting temporary audio after processing. Admins can manage prompts, filler words, and thresholds.

2. Problem and product decision

Generic recorders tell a speaker what they said but not how they delivered it. Human coaching is valuable but not always available. The product decision was to make the first version narrow and measurable: turn one rehearsal into a repeatable loop of prompt → recording → objective feedback → another attempt.

The product intentionally does not claim to judge a speaker's personality, confidence, intelligence, or overall presentation quality. It reports measurable audio/transcript signals and gives friendly, specific coaching language.

3. Scope delivered

Delivered

- Account registration and JWT login/refresh.
- User and admin access control.
- Prompt categories, search, language and difficulty filters.
- Browser recording and audio-file upload.
- File extension, size, and duration validation.
- Background processing with status polling.
- faster-whisper transcription with a mock fallback.
- English and Bangla configuration paths.
- Overall/windowed WPM and configurable target pace.
- Pause detection and short/dramatic/long classification.
- Global and personal filler-word lists.
- Transcript with word timings and filler timestamps.
- Breakdown, recording history, deletion, and progress endpoints.
- Django admin panel for prompts, users, settings, and global words.
- React pages for authentication, practice, recording, breakdown, history, progress, filler words, and admin settings.
- Responsive/accessibility-minded UI with reduced-motion and no-WebGL fallbacks.

Not delivered in this release

Pitch variation, camera-based eye contact or gesture analysis, live peer rehearsal, audience-type pacing modes, humor scoring, open-ended-question analysis, password reset, email verification, and a mobile APK.

4. Architecture and tools

Tools and technologies used

Area	Tools
Frontend	React 19, TypeScript, Vite, Tailwind CSS, Framer Motion, Three.js, Recharts, ESLint
Backend	Python, Django, Django REST Framework, SimpleJWT, django-cors-headers
Speech	faster-whisper, av, local model execution, mock transcription fallback
Data	SQLite for local/demo use; Django ORM and migrations
Testing	Django test runner, service tests, endpoint tests, upload/permission tests, `try_api.py`, frontend TypeScript build and ESLint
Documentation	Markdown, DOCX, diagrams, PDF handoff documents
Collaboration	Git/GitHub workflow, branches, conventional commits, pull requests, project log

No paid speech API or API key is required by the current implementation.

5. Development journal

13 October 2026 — backend skeleton

The backend structure was implemented around Django REST Framework, five application areas, plain-Python analysis services, and tests. The implementation followed the system design. AI assistance was used during backend implementation and is documented openly in the project log. The initial checks covered the API flow and 118 automated tests. Real Whisper transcription remained an open verification item.

14 October 2026 — cross-machine setup and test console

Exact package pins caused installation failures on Python 3.9. The solution was to use compatible version ranges and a one-command runner that creates the virtual environment, installs dependencies, migrates the database, optionally downloads the model, starts the server, and opens the test page. A browser test console was added to exercise registration, login, prompt selection, upload, status, results, fillers, history, and deletion.

The mock engine was deliberately retained as a graceful fallback when the model is unavailable. The UI and transcript identify the engine so a user does not confuse demo output with real transcription.

16 October 2026 — audio decoder failure

Real uploads failed with `metadata_errors` when faster-whisper was used with `av` 19.x. The issue was reproduced across several versions. The solution was to cap `av` below 19 in `requirements-whisper.txt`, add an audio-decoder readiness check, and preserve the underlying reason in the user-facing error. This decision is important because speech dependencies can break at their integration boundary even when each package installs successfully.

Frontend implementation phase

The frontend was built against the captured API contract rather than guessed endpoints. The API client centralizes trailing slashes, JWT refresh, error normalization, 202 upload handling, two-second status polling, five-minute timeout, and parallel result requests. The UI reads pace thresholds from the backend response and hides admin features unless `is_admin` is true. A contract-faithful in-browser demo backend makes the interface reviewable when the API is unavailable.

26 October 2026 — submission package

The handoff package was consolidated into a root README, Markdown PRD, Markdown System Design, full project report, deployment guide, deployed-link sheet, credentials/env guide, and PDF versions of the main submission documents. No application source code was changed for this documentation pass. Secrets remain represented by private environment variables and safe templates; only evaluation/demo account information already present in the project is documented.

6. Important decisions and reasons

Decision	Reason
Use JWT access/refresh tokens	Works cleanly across a separate frontend and API without server-side browser cookies.
Keep API under `/api/v1/`	Gives the frontend a stable versioned contract.
Return HTTP 202 for upload	Transcription can take time; the user gets immediate acknowledgement and polls status.
Process in a background thread	Keeps the demo responsive without adding a queue dependency.
Keep a mock engine	Local review still works without a model download, while the engine label prevents misleading results.
Store settings snapshots	Historical analyses remain explainable after thresholds change.
Delete audio after processing	Reduces retained sensitive data; transcript and measurements are sufficient for progress.
Use SQLite for the academic/demo build	Zero setup and adequate for local evaluation; PostgreSQL is the production follow-up.
Use configurable filler words	Words such as “like” can be legitimate in context and language needs differ.
Use a friendly coaching tone	The product should encourage deliberate practice rather than shame mistakes.
Keep camera analysis out of MVP	It requires a different privacy, computer-vision, and validation scope.

7. Challenges and solutions

Challenge	Solution
Package compatibility on different Python versions	Version ranges, a setup runner, and a documented Python 3.10–3.13 recommendation.
Whisper model unavailable or too heavy	Automatic mock fallback with visible engine status; strict mode remains available.
`av` 19 incompatibility	Cap `av` below 19 and test decoder readiness before transcription.
Slow speech processing	202 response, background job, status endpoint, frontend polling, timeout, and failure reason.
Different frontend/backend origins	Environment-driven API base and explicit CORS configuration.
Protecting user data	Owner-filtered querysets, not-found behavior for other users, JWT permissions, and cleanup of audio.
WebGL or motion support varies	Lazy-loaded Three.js scenes and static/reduced-motion fallbacks.
Documentation drift	Frozen API contract notes, companion Markdown documents, and a project log.

8. Verification record

The project records the following checks in its existing log and handoff documentation:

- 136 automated tests covering services, endpoints, permissions, uploads, errors, and failure handling.
- An end-to-end `try_api.py` flow covering registration, login, upload, polling, transcript, analysis, progress, fillers, settings, CORS, and deletion.
- Browser-console walkthroughs for the main user journey.
- Frontend build and lint scripts are defined as `npm run build` and `npm run lint`.
- The codebase has a mock path for environments where the Whisper model cannot be downloaded.

For final submission, run the local commands below on the machine that has the intended Python installation and record the result:

```
cd backend
python run.py test
python run.py check

cd ../frontend
npm run build
npm run lint
```

The current workspace's virtual-environment launcher points to an unavailable original Python installation, so the Python checks must be rerun after recreating the environment with an installed Python 3.10–3.13 interpreter.

9. Security and privacy handoff

Passwords are hashed by Django, not stored as plain text. The repository ignores `.env`, SQLite, audio, model files, virtual environments, and build output. The current project uses no third-party API key. Production deployment must use a new `SECRET_KEY`, `DEBUG=0`, exact `ALLOWED_HOSTS`, exact frontend CORS origin, rotated demo passwords, HTTPS, and private host environment variables.

The demo credentials in `CREDENTIALS_AND_ENV.md` are not production credentials. If this repository becomes public, rotate them first and share the new values privately.

10. Deployment status and handoff

The source currently contains local run instructions and a free deployment procedure. There is no verified public URL in the workspace yet, and no APK because the deliverable is a web application. After creating the GitHub repository and deploying, update `DEPLOYED_LINKS.md` with the repository, frontend, backend, and admin URLs.

11. Future work

1. Replace in-process threads with a durable queue and retry policy.
2. Move from SQLite to PostgreSQL and use durable audio/object storage only where necessary.
3. Add a production WSGI server and monitoring.
4. Measure real-speaker accuracy for English and Bangla.
5. Add pitch/vocal-variety, question-type, and camera-based features only after privacy and accuracy validation.
6. Add password reset, email verification, and account lifecycle controls.

12. Conclusion

Backstage delivers a complete academic/demo MVP around a clear practice loop. The strongest engineering decisions are the versioned API contract, asynchronous analysis, visible mock fallback, configurable thresholds, owner-level data protection, and a documentation trail that records both successful

work and unresolved limits. The remaining handoff task is operational: create the GitHub remote, deploy using the documented free path, test the live URLs, and record those URLs privately and in the link sheet.